

Selection Algorithms

Learning Objectives

- Implement selection (conditional) logic using IF-THEN-ELSE constructs
- Design multi-way selection using nested and chained conditions
- Apply selection in algorithms requiring decision-making based on criteria

Engage (5-7 minutes)

Play a “20 Questions” game where students narrow down an object through yes/no questions. Each question eliminates half the possibilities—this is selection logic in action. Discuss how computers make decisions the same way: evaluating conditions and branching to different code paths.

Explore (10-12 minutes)

In groups, students design an algorithm for a chatbot that responds differently based on user input: greeting, query about weather, complaint, or unknown input. They create a decision tree on paper, then implement it in pseudocode using IF-ELSEIF-ELSE chains. Groups test with edge cases (empty input, ambiguous commands).

Explain (10-12 minutes)

Selection algorithms use conditional statements to choose between alternative paths based on evaluated conditions. The basic forms are: single-branch (IF-THEN), dual-branch (IF-THEN-ELSE), and multi-way (IF-ELSEIF-ELSE or CASE/WHITCH). Conditions are Boolean expressions that evaluate to true or false. Nested selection allows complex decision logic but increases cognitive load—flattening with guard clauses improves readability.

Elaborate (8-10 minutes)

Examine real-world selection systems: credit scoring (IF credit_score > 700 THEN approve ELSE IF score > 600 THEN review ELSE reject), medical triage algorithms, and game AI behavior trees. Discuss the importance of condition ordering—when conditions overlap, the first matching branch executes. Analyze how short-circuit evaluation in programming affects performance and logic.

Evaluate (5-8 minutes)

Students implement a tax calculator using selection: different tax brackets based on income. Assessment checks correct boundary handling (e.g., exactly at threshold values), coverage of all cases, and absence of unreachable code. Extension: handle invalid inputs gracefully.